

BATCH No:MAI273

ANIMAL INTRUSION DETECTION USING YOLO V9

*Major project report submitted
in partial fulfillment of the requirement for award of the degree of*

**Bachelor of Technology
in
Computer Science & Engineering**

By

K TEJASWI	(21UECM0108)	(20190)
SANAGAVARAPU SHALINI	(21UECM0213)	(20217)
MALLAVARAPU RAMYA SREE	(21UECS0354)	(19216)

*Under the guidance of
Dr. Arunkumar M.S, M.E.,Ph.D.,
ASSOCIATE PROFESSOR*



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF
SCIENCE AND TECHNOLOGY**

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

**Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA**

May 2025

BATCH No:MAI273

ANIMAL INTRUSION DETECTION USING YOLO V9

*Major project report submitted
in partial fulfillment of the requirement for award of the degree of*

**Bachelor of Technology
in
Computer Science & Engineering**

By

K TEJASWI	(21UECM0108)	(20190)
SANAGAVARAPU SHALINI	(21UECM0213)	(20217)
MALLAVARAPU RAMYA SREE	(21UECS0354)	(19216)

*Under the guidance of
Dr. Arunkumar M.S, M.E.,Ph.D.,
ASSOCIATE PROFESSOR*



**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN DR. SAGUNTHALA R&D INSTITUTE OF
SCIENCE AND TECHNOLOGY**

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

**Accredited by NAAC with A++ Grade
CHENNAI 600 062, TAMILNADU, INDIA**

May 2025

CERTIFICATE

It is certified that the work contained in the project report titled "ANIMAL INTRUSION DETECTION USING YOLO V9" by "K TEJASWI & (21UECM0108), SANAGAVARAPU SHALINI & (21UECM0213), MALLAVARAPU RAMYA SREE & (21UECS0354)" has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

Signature of Supervisor

Dr. Arunkumar M.S

Associate Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

Signature of Head/Assistant Head of the Department

Dr. N. Vijayaraj/Dr. M. S. Murali dhar

Professor & Head/ Professor & Assistant Head

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

Signature of the Dean

Dr. S P. Chokkalingam

Professor & Dean

School of Computing

Vel Tech Rangarajan Dr. Sagunthala R&D

Institute of Science and Technology

DECLARATION

We declare that this written submission represents my ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(K TEJASWI)

Date: / /

(SANAGAVARAPU SHALINI)

Date: / /

(MALLAVARAPU RAMYA SREE)

Date: / /

APPROVAL SHEET

This project report entitled "ANIMAL INTRUSION DETECTION USING YOLO V9" by K TE-JASWI (21UECM0108), SANAGAVARAPU SHALINI (21UECM0213), MALLAVARAPU RAMYA SREE (21UECS0354) is approved for the degree of B.Tech in Computer Science & Engineering.

Examiners

Supervisor

Dr. Arunkumar. M.S, M.E.,Ph.D.,
ASSOCIATE PROFESSOR.

Date: / /

Place:

ACKNOWLEDGEMENT

We express our deepest gratitude to our **Honorable Founder Chancellor and President Col. Prof. Dr. R. RANGARAJAN B.E. (Electrical), B.E. (Mechanical), M.S(Automobile), D.Sc., and Foundress President Dr. R. SAGUNTHALA RANGARAJAN M.B.B.S., Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, for her blessings.**

We express our sincere thanks to our respected Chairperson and Managing Trustee **Mrs. RANGARAJAN MAHALAKSHMI KISHORE, B.E., Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology, for her blessings.**

We are very much grateful to our beloved **Vice Chancellor Prof. Dr. RAJAT GUPTA**, for providing us with an environment to complete our project successfully.

We record indebtedness to our **Professor & Dean , School of Computing, Dr. S P. CHOKKALINGAM, M.Tech., Ph.D., & Professor & Associate Dean , School of Computing, Dr. V. DHILIP KUMAR, M.E., Ph.D.,** for immense care and encouragement towards us throughout the course of this project.

We are thankful to our **Professor & Head, Department of Computer Science & Engineering, Dr. N. VIJAYARAJ, M.E., Ph.D., and Associate Professor & Assistant Head, Department of Computer Science & Engineering, Dr. M. S. MURALI DHAR, M.E., Ph.D.,** for providing immense support in all our endeavors.

We also take this opportunity to express a deep sense of gratitude to our **Internal Supervisor, Dr. ARUNKUMAR. M.S, M.E., Ph.D.,** for his cordial support, valuable information and guidance, he helped us in completing this project through various stages.

A special thanks to our **Project Coordinators Dr. SADISH SENDIL MURUGARAJ, Professor, Dr. S. RAJAPRAKASH, M.E., Ph.D., Mr. V. ASHOK KUMAR, B.E., M.Tech.,** for their valuable guidance and support throughout the course of the project.

We thank our department faculty, supporting staff and friends for their help and guidance to complete this project.

K TEJASWI	(21UECM0108)
SANAGAVARAPU SHALINI	(21UECM0213)
MALLAVARAPU RAMYA SREE	(21UECS0354)

ABSTRACT

Animal intrusion in human-populated areas poses significant threats to both wildlife and human safety, necessitating efficient monitoring and detection systems. This project focuses on developing an animal intrusion detection model using YOLOv9, a deep learning object detection algorithm. The suggested approach uses a specially trained YOLOv9 model to precisely identify and categorize different animal species. To ensure robust and accurate results, the training involves a diverse dataset of images including different types of animals in varied environments. The overall aim is to facilitate high accuracy and real-time detection for proactive action in preventing human-wildlife conflicts. Experimental results confirm that the proposed model successfully detects animals with state-of-the-art accuracy and significantly lower inference time than its predecessors of YOLO. This project further provides a groundwork for future connection with hardware systems (for instance, surveillance cameras and drones) to automate monitoring of close ecosystems in forest edges, farming fields, and urban settlements.

Keywords: YOLO V9, Object Detection, Wildlife Monitoring, Intrusion Prevention, Deep Learning

LIST OF FIGURES

4.1	Architecture Diagram of Animal Intrusion Detection System . .	15
4.2	Data Flow Diagram	16
4.3	Sequence Diagram	17
4.4	Collaboration Diagram	18
4.5	Activity Diagram	19
4.6	Sample of Dataset	22
4.7	Pre-Processing the Dataset	23
4.8	Training the Model	24
4.9	Testing the Model	25
5.1	Animal Dataset	28
5.2	Detection of Animals on User Interface	29
5.3	Result of Unit Test	31
5.4	Result of Integration Test	32
5.5	Result of Functional Test	34
5.6	Test Image	35
6.1	Comparison Graph	39
9.1	Plagiarism	46

LIST OF TABLES

4.1	Dataset Structure	21
6.1	Comparison of Different Models with Proposed Model	38

LIST OF ACRONYMS AND ABBREVIATIONS

DL	Deep Learning
FN	False Negatives
FP	False Positives
IOC	Intersection over Union
MAP	Mean Average Precision
TP	True Positives
YOLO V9	You Only Look Once Version 9

TABLE OF CONTENTS

	Page.No
ABSTRACT	v
LIST OF FIGURES	vi
LIST OF TABLES	vii
LIST OF ACRONYMS AND ABBREVIATIONS	viii
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Background	2
1.3 Objective	3
1.4 Problem Statement	3
2 LITERATURE REVIEW	5
2.1 Existing System	6
2.2 Related Work	7
2.3 Research Gap	7
3 PROJECT DESCRIPTION	9
3.1 Existing System	9
3.2 Proposed System	10
3.3 Feasibility Study	10
3.3.1 Economic Feasibility	11
3.3.2 Technical Feasibility	11
3.3.3 Social Feasibility	12
3.4 System Specification	13
3.4.1 Tools and Technologies Used	13
3.4.2 Standards and Policies	14
4 SYSTEM DESIGN AND METHODOLOGY	15
4.1 System Architecture	15

4.2	Design Phase	16
4.2.1	Data Flow Diagram	16
4.2.2	Sequence Diagram	17
4.2.3	Collaboration diagram	18
4.2.4	Activity Diagram	19
4.3	Algorithm & Pseudo Code	19
4.3.1	Algorithm	19
4.3.2	Pseudo Code	20
4.4	Module Description	21
4.4.1	Collection of Dataset	21
4.4.2	Pre-Processing the Dataset	22
4.4.3	Training the Model	23
4.4.4	Testing the Model	24
4.5	Steps to execute/run/implement the project	25
4.5.1	Setup Environment	25
4.5.2	Download Dataset	26
4.5.3	Data Preprocessing	26
4.5.4	Model Configuration	26
4.5.5	Model Training	26
4.5.6	Model Evaluation	26
4.5.7	Results and Analysis	26
4.5.8	Model Deployment	27
5	IMPLEMENTATION AND TESTING	28
5.1	Input and Output	28
5.1.1	Input Design	28
5.1.2	Output Design	29
5.2	Testing	29
5.2.1	Testing Strategies	29
5.2.2	Performance Evaluation	35
6	RESULTS AND DISCUSSIONS	36
6.1	Efficiency of the Proposed System	36
6.2	Comparison of Existing and Proposed System	37
6.3	Comparative Analysis-Table	38
6.4	Comparative Analysis-Graphical Representation and Discussion . .	39

7	CONCLUSION AND FUTURE ENHANCEMENTS	40
7.1	Summary	40
7.2	Limitations	41
7.3	Future Enhancements	41
8	SUSTAINABLE DEVELOPMENT GOALS (SDGs)	43
8.1	Alignment with SDGs	43
8.2	Relevance of the Project to Specific SDG	44
8.3	Potential Social and Environmental Impact	45
8.4	Economic Feasibility	45
9	PLAGIARISM REPORT	46
10	SOURCE CODE	47
10.1	Source Code	47
	References	50

Chapter 1

INTRODUCTION

1.1 Introduction

Human-wildlife conflict is an increasing problem worldwide due to urban expansion, deforestation and habitat encroachment. Wild animals often enter human settlements in search of food and shelter, leading to damage crops, property and in some cases threats to human safety. This growing issue has emphasized the need for efficient and automated animal intrusion detection systems to ensure both human and wildlife safety. This project aims to develop an AI-powered Animal Intrusion Detection System using YOLOv9 (You Only Look Once), an advanced deep-learning object detection model. YOLO is widely recognized for its high-speed and accurate real-time detection capabilities, making it suitable for continuous monitoring and immediate alerts. The model will be trained on a dataset containing images of various animals, allowing it to identify and classify species with high precision.

Once deployed, the system will process images to detect animal presence, reducing the risk of human-wildlife conflict. The detection results can be applied in wildlife monitoring, farm protection, and security surveillance, helping authorities and farmers take timely action to prevent damage and ensure safety. To enhance usability, a user-friendly graphical interface will be developed, allowing seamless interaction with the system. The integration of computer vision, artificial intelligence, and deep learning will enable real-time monitoring and intelligent decision-making, making this project a valuable solution for protecting both human communities and wildlife. By leveraging cutting-edge AI technologies, this project aims to create a cost-effective, scalable, and efficient solution that contributes to environmental conservation while safeguarding human interests.

1.2 Background

Human-wildlife conflict is a growing global concern, primarily caused by rapid urbanization, deforestation, and habitat encroachment. As natural habitats shrink due to human activities, wild animals are increasingly forced to enter human settlements in search of food, water, and shelter. This often leads to destruction of crops, damage to property, loss of livestock and even direct threats to human safety. Farmers, rural communities and conservationists face significant challenges in managing these conflicts, which not only cause economic losses but also impact biodiversity. Traditional methods such as fencing, scare tactics and manual patrolling have been employed to mitigate these issues. However, these approaches are often ineffective, labor-intensive and expensive, especially in large or remote areas. Additionally, improper handling of animal intrusions can lead to retaliatory killings, further endangering wildlife populations. Therefore, there is a need for a modern, automated and efficient solution that ensures the safety of both humans and animals. In recent years, Artificial Intelligence (AI) and computer vision technologies have revolutionized the field of wildlife monitoring. Deep learning-based object detection models, such as YOLOv9 (You Only Look Once), offer a high-speed, real-time solution for identifying and classifying animal intrusions with remarkable accuracy. These systems process detect wildlife movement and send alerts , allowing timely intervention to prevent damage.

By integrating AI, machine learning and automated surveillance systems, this project aims to develop an advanced Animal Intrusion Detection System that can effectively monitor wildlife activity in farmlands, forests and residential areas. Such a system can provide a cost-effective, scalable, and environmentally sustainable approach to mitigating human-wildlife conflict while promoting peaceful coexistence between humans and nature. The implementation of such intelligent monitoring systems has the potential to benefit not only farmers and local communities but also conservation organizations and government agencies. By reducing conflicts and enabling proactive wildlife management, this technology can contribute to sustainable development, biodiversity conservation, and improved human-wildlife interactions. Moreover, as AI-powered solutions continue to evolve, they hold the promise of making wildlife protection more effective and accessible on a global scale.

1.3 Objective

The primary objective of this project is to develop a real-time animal intrusion detection system using YOLOv9 that can accurately identify and classify animals in diverse environments. By leveraging deep learning and computer vision, the system aims to detect animal presence with high precision, minimizing false positives and negatives. The model will be trained on a diverse dataset covering different species, environmental conditions, and terrains to ensure robustness and adaptability. This will enhance detection accuracy and enable efficient monitoring of animal movements in farmlands, forests, and human settlements. Furthermore, the model's performance will be evaluated using appropriate metrics such as precision, recall, and mAP (mean Average Precision) to ensure reliability in real-world applications.

Another key objective is to optimize the system for efficiency by ensuring real-time processing with minimal computational cost. The system will be designed to function effectively on edge devices or cloud-based platforms, making it scalable and accessible for various field applications. Additionally, a user-friendly interface will be developed to allow seamless interaction with the trained model, providing alerts and insights to users in case of animal intrusion. The project aims to create a cost-effective and practical solution for wildlife monitoring, farm protection, and security surveillance, ultimately reducing human-wildlife conflict and promoting coexistence between humans and animals.

1.4 Problem Statement

The primary objective of this project is to develop a real-time animal intrusion detection system using YOLOv9 that can accurately identify and classify animals in diverse environments. By leveraging deep learning and computer vision, the system aims to detect animal presence with high precision, minimizing false positives and negatives. The model will be trained on a diverse dataset covering different species, environmental conditions, and terrains to ensure robustness and adaptability. This will enhance detection accuracy and enable efficient monitoring of animal movements in farmlands, forests, and human settlements. Furthermore, the model's

performance will be evaluated using appropriate metrics such as precision, recall, and mAP to ensure reliability in real-world applications.

This project aims to develop a deep learning-based animal intrusion detection system using YOLOv9 to enhance accuracy and efficiency in identifying and classifying animals in real-time. By leveraging computer vision and AI, the system will enable automated surveillance, reducing dependency on human intervention while ensuring faster and more reliable detection. The proposed system will be trained on a diverse dataset to recognize various animal species in different environmental conditions, making it highly adaptable for field applications. This solution can be beneficial in agriculture, wildlife conservation, and security surveillance, paving the way for advancements in AI-driven monitoring systems that help mitigate human-wildlife conflicts effectively.

Chapter 2

LITERATURE REVIEW

- [1] J. Chappidi et al., in 2024 suggested a new method for detecting animals that uses a cascaded YOLOv8 approach with adaptive preprocessing and feature extraction for wildlife detection, contributing to biodiversity monitoring and conservation efforts, and suggesting future improvements with advanced deep learning models and diverse datasets.
- [2] K. M, B. V et al., in 2024 proposed an Animal Intrusion Detection System using YOLOv8, focusing on precise animal identification and real time alert mechanisms to mitigate human-wildlife conflicts, protect crops, and support biodiversity conservation, leveraging YOLOv8's enhanced accuracy and speed for effective monitoring in agricultural fields.
- [3] Y. Zhang et al., in 2024 enhanced YOLOv5 for safety risk detection in power substations by integrating Mosaic-9 data augmentation, CA attention module, and BiFPN feature fusion, achieving a 4.6high-precision detection of small animal intrusions and other safety risks.
- [4] A. P E et al., in 2023 developed an Animal Intrusion Detection System using YOLO for image based detection and a deep learning-based audio model to identify wild animal sounds, enabling real time alerts to authorities to mitigate human-wildlife conflicts and prevent resource damage.
- [5] R. Kothai et al., in 2023 explored the YOLO algorithm for animal intrusion detection and crop protection, leveraging its grid-based object detection approach to efficiently localize and classify animals in a single pass.
- [6] M. V. K. Rao et al., in 2023 proposed a machine learning-based monitoring system using YOLO for farm animal intrusion detection, providing real-time alerts to mitigate crop losses and reduce the financial burden on farmers.
- [7] J. J. Daniel Raj et al., in 2023 developed an IoT based animal intrusion detection system using YOLO and Raspberry Pi to identify animals near forest highways, providing real-time alerts to drivers to reduce accidents and ensure wildlife safety.
- [8] A. D. Shetty et al., in 2023 proposed a YOLO based animal detection system for wildlife conservation and intrusion prevention, comparing YOLOv5 and YOLOv8,

with YOLOv8 achieving higher accuracy for real-time monitoring and security applications.

[9] R. Sumathi et al., in 2022 proposed a real-time farmland surveillance system using YOLO for animal intrusion detection, providing geo-location alerts and employing non-lethal repellents to mitigate human-animal conflicts without harming wildlife.

[10] P. Muragod et al., in 2022 developed an animal intrusion detection system using deep learning models, with YOLOv5 achieving 93% integrating feature extraction and SMS alerts to landowners for real-time monitoring and crop protection.

2.1 Existing System

Traditional animal intrusion detection methods primarily rely on static surveillance systems, motion sensors, and GPS-based tracking. These approaches, while effective to some extent, have several limitations that make them unsuitable for real-time, large-scale applications.

1. **Traditional Surveillance Systems**– Conventional systems use CCTV cameras and manual monitoring, where security personnel or farmers analyze video footage to detect animal activity. These methods lack real-time detection capabilities and require significant human intervention, making them inefficient for monitoring large areas or remote locations.

2. **Infrared and Motion Sensors** – Some wildlife monitoring systems use infrared or motion sensors to detect movement. However, these systems often produce high false-positive rates, as they mistake wind, foliage movement, or temperature changes for actual animal intrusions, leading to unreliable detection.

3. **Geo-Fencing with GPS Collars** – In certain cases, animals are tagged with GPS collars to track their movements and issue alerts when they cross predefined boundaries. While this method provides accurate tracking, it requires animals to be physically tagged, which is impractical for large wildlife populations and does not provide real-time monitoring of untagged intruders.

Additionally, previous deep learning-based models such as YOLOv8 and older versions have limitations in accuracy and processing speed, making them less suitable for real-time applications. These challenges highlight the need for an advanced, AI-powered system like YOLOv9, which offers higher accuracy, real-time processing, and improved adaptability to diverse environmental conditions.

2.2 Related Work

Several research studies and projects have explored the use of computer vision and deep learning for wildlife monitoring and animal intrusion detection. These works demonstrate the growing adoption of AI-based solutions over traditional surveillance methods.

1. Deep Learning for Wildlife Detection – Recent studies have implemented CNN-based models such as YOLO and Faster R-CNN for detecting and classifying animals in video feeds. While these models have shown promising results, their deployment often requires high computational resources, making real-time processing a challenge in low-power or edge computing environments.

2. Motion-Based Detection Systems – Some researchers have explored motion detection algorithms combined with image processing techniques to identify animal movements in agricultural or protected areas. However, these systems are prone to false positives due to wind, lighting changes, and non-animal movements.

3. IoT and Sensor-Based Animal Detection – Various studies have integrated IoT sensors, thermal cameras, and GPS-based tracking to monitor wildlife activities. While these solutions provide continuous tracking, they require physical tagging of animals or expensive sensor networks, limiting their feasibility for widespread implementation.

4. YOLO-Based Animal Recognition – Previous versions of YOLO, such as YOLOv3 to YOLOv8, have been used for animal detection tasks. Although these models offer good accuracy, they lack optimized real-time performance for large-scale monitoring. Additionally, older YOLO models struggle with varying environmental conditions, different lighting scenarios, and diverse animal species.

2.3 Research Gap

Despite advancements in computer vision-based animal detection, several challenges remain unaddressed in existing research and implementations. Traditional motion detection systems and sensor-based methods have shown limitations in accuracy, adaptability, and real-time processing, necessitating the development of more efficient AI-driven solutions. Many previous studies utilizing models like YOLOv3 to YOLOv8 or other deep learning frameworks struggle with low accuracy in complex environments. Factors such as poor lighting, occlusion, and background

noise often result in false detections. Additionally, high computational costs make real-time processing difficult, limiting large-scale deployments, especially in remote or resource-constrained areas. Another major gap in existing research is the lack of generalization across species. Many animal detection systems are trained on limited datasets, making them ineffective in recognizing multiple species under diverse terrains and climatic conditions. Furthermore, most previous implementations rely on hardware-based solutions like infrared sensors, GPS collars, or IoT devices, which increase costs, require regular maintenance, and pose deployment challenges. These limitations highlight the need for a purely software-based approach that eliminates hardware dependencies while ensuring high detection accuracy.

This project aims to bridge these gaps by utilizing YOLOv9, a more advanced and optimized deep-learning model that enhances speed, accuracy, and real-time processing. By training the model on diverse datasets and optimizing it for low-latency inference, this project ensures higher accuracy in animal classification without relying on additional hardware. The software-only approach not only reduces implementation costs but also makes the system more scalable and adaptable for practical applications in agriculture, wildlife monitoring, and security surveillance.

Chapter 3

PROJECT DESCRIPTION

3.1 Existing System

The existing animal intrusion detection systems primarily rely on manual monitoring, motion-based detection, and traditional image processing techniques. Security personnel or farmers often depend on CCTV footage and manual surveillance, which is not only time-consuming but also prone to human errors. While some software-based solutions utilize basic motion detection algorithms, these methods fail to differentiate between animals, humans, and other environmental movements like swaying trees or changing light conditions. As a result, these systems often generate false alarms, reducing their effectiveness in real-world applications.

Another significant drawback of current systems is their lack of adaptability and real-time processing capabilities. Many traditional approaches do not incorporate deep learning models, which results in low detection accuracy and an inability to recognize various animal species across different terrains and environmental conditions. Moreover, non-AI-based systems require constant supervision, making them impractical for large-scale or remote applications. These limitations highlight the need for an advanced, AI-driven software solution that can automate the process, enhance accuracy, and reduce dependency on human intervention.

Disadvantages of the Existing System:

- High dependency on manual surveillance, leading to inefficiency.
- Frequent false alarms due to the inability to distinguish between animals and other objects.
- Limited adaptability to different environmental conditions.
- Lack of real-time processing and intelligent classification of different species.
- Not scalable for large areas such as forests and agricultural fields.

3.2 Proposed System

The proposed system aims to develop an AI-powered animal intrusion detection model using YOLOv9, a cutting-edge deep learning algorithm. Unlike traditional systems, this model will be fully automated and software-based, requiring no manual monitoring or additional hardware components. By leveraging computer vision and deep learning, the system will be able to accurately detect and classify different animal species in real-time. The model will be trained on a diverse dataset to ensure high recognition accuracy across various lighting conditions, terrains, and environments.

One of the key advantages of this proposed system is its scalability and efficiency. The YOLOv9 model is designed to process images at high speed while maintaining accuracy, making it suitable for applications such as wildlife monitoring, farm security, and ecological research. Since this system is purely software-based, it eliminates the need for physical sensors or alarms, making it a cost-effective and flexible solution. The system can be deployed on cloud-based or edge-computing platforms, enabling seamless integration into existing security frameworks.

Advantages of the Proposed System:

- Highly accurate and real-time detection using YOLOv9 deep learning technology.
- Adaptable across various environmental conditions and terrains.
- Scalable and cost-effective, making it accessible for large-scale applications.
- Reduces human intervention, increasing efficiency and reliability.

3.3 Feasibility Study

The feasibility of this project is analyzed in some key aspects: technical, operational, and economic viability. From a technical standpoint, the proposed system leverages deep learning and computer vision technologies, specifically the YOLOv9 model, to detect and classify animals accurately. From an operational perspective, this system is developed to be scalable and user-friendly. Economically, this project

presents a cost-effective solution by eliminating the need for expensive hardware installations.

3.3.1 Economic Feasibility

The economic feasibility of the proposed animal intrusion detection system is highly favorable, primarily due to its software implementation. Unlike traditional systems that require costly hardware components such as motion sensors, infrared cameras, GPS collars, or alarm systems, this project relies solely on machine learning and computer vision algorithms. This significantly reduces the initial investment and eliminates the need for purchasing, installing, or maintaining external hardware. The use of YOLOv9, an open-source deep learning model, further contributes to cost savings as it avoids licensing fees while providing state-of-the-art accuracy and performance.

Additionally, the system can be implemented on standard computing devices or cloud-based platforms, both of which are economically viable for individuals, farmers, or wildlife authorities. Once deployed, the automation aspect of the software reduces the need for continuous human monitoring, which in turn lowers labor costs and increases long-term savings. Since the model can be retrained and updated with new data, there's no need for rebuilding or purchasing new systems, making it a sustainable and scalable investment. The system's ability to prevent crop damage, reduce economic losses, and improve resource allocation also contributes indirectly to the financial benefits. Overall, the proposed solution is economically practical, making advanced AI-based wildlife monitoring affordable and accessible to a wider audience.

3.3.2 Technical Feasibility

The technical feasibility of the proposed animal intrusion detection system is strongly supported by the maturity of current machine learning, computer vision, and deep learning technologies. The system is built using YOLOv9, a state-of-the-art object detection algorithm known for its speed, accuracy, and efficiency in real-time scenarios. This algorithm is highly capable of detecting and classifying multiple animal species from images or video streams, even under varying lighting and environmental conditions. The model is trained using a diverse and annotated

dataset, enabling it to generalize well across different terrains, species, and image qualities.

From an implementation standpoint, the project leverages Python programming, along with popular libraries such as OpenCV, and TensorFlow, which are widely used and supported in the AI community. These tools offer extensive documentation and community support, reducing technical risk and increasing reliability. The software can run on standard laptop or desktop hardware, and it can also be scaled to cloud platforms for larger applications. This further enhances the technical feasibility by keeping the system modular, adaptable, and easy to deploy across different environments. Overall, the project is technically sound and implementable with current technologies, ensuring high performance and practical applicability.

3.3.3 Social Feasibility

The social feasibility of this project is highly promising, as it addresses a pressing issue that affects both rural and semi-urban communities human-wildlife conflict. With frequent incidents of wild animals intruding into farms, residential areas, and restricted zones, there is a growing need for an effective, real-time solution that can mitigate risks to human lives, crops, and property. The proposed system, being entirely software-based, is non-invasive and does not harm the animals or the environment, making it socially acceptable and ethically sound. It fosters co-existence between humans and wildlife by enabling early detection and timely intervention, thereby reducing panic-driven reactions such as animal culling or harmful deterrents.

Furthermore, this system can empower local communities, especially farmers and forest dwellers, by providing them with a reliable and affordable technological solution. Since it eliminates the need for constant human monitoring, it reduces stress and labor, contributing positively to their quality of life. The user-friendly interface ensures that even individuals with limited technical knowledge can use the system effectively. In the long term, widespread adoption of such intelligent systems can promote awareness and education about wildlife behavior and conservation, contributing to broader social and ecological sustainability. Hence, this project is not only technically and economically viable but also socially beneficial and community-friendly.

3.4 System Specification

Hardware Specification

- **Graphics Processing Unit (GPU):**
 - Intel(R) Iris(R) Xe Graphics
- **Central Processing Unit (CPU):**
 - 12th Gen Intel(R) Core(TM) i5-1235U
- **Memory (RAM):**
 - DDR4 (Minimum 32GB)
- **Storage:**
 - Solid State Drive (SSD)

Software Specification

- **Programming Language:**
 - Python (version 3.6 or higher)
- **Integrated Development Environment (IDE):**
 - Visual Studio Code or Jupyter Notebooks
- **Cloud Platform:**
 - Google Colab

3.4.1 Tools and Technologies Used

- **Python** – Used as the primary programming language for model training and preprocessing.
- **Google Colab** – Utilized as the development environment with GPU support for training the YOLOv9 model.
- **YOLOv9** – State-of-the-art object detection algorithm used to train and detect animals in the dataset.
- **PyTorch** – Deep learning framework used for implementing and training the YOLOv9 model.

- **OpenCV** – Used for image processing tasks and visualization of detection results.
- **Matplotlib & Seaborn** – Libraries used for data visualization during exploration and analysis.
- **Flask & Visual Studio Code** – Used for creating a simple and interactive user interface for testing the trained model.

3.4.2 Standards and Policies

Google Colab

Google Colab, short for Colaboratory, is an online platform provided by Google that offers a cloud-based environment for running Python code, particularly popular for its integration with Jupyter Notebooks. Unlike local environments, Google Colab operates entirely in the cloud, enabling collaborative coding and resource-intensive tasks without the need for extensive hardware specifications. While Google Colab doesn't adhere to a specific ISO/IEC standard like other applications, its usage aligns with general cybersecurity and data protection principles. Users are encouraged to follow best practices in securing their notebooks, such as restricting access and managing data privacy according to their organizational or institutional standards.

Standard Used: ISO/IEC 27001

Visual Studio Code

Visual Studio Code (VS Code) is a source-code editor developed by Microsoft, known for its versatility and support for various programming languages. While it doesn't adhere to a specific ISO/IEC standard, Microsoft, the developer of Visual Studio Code, maintains a commitment to security and privacy in its software development practices. Therefore, while Visual Studio Code itself may not have a direct standard, users can benefit from Microsoft's broader adherence to industry-recognized standards for information security and data protection. It's advisable for users to implement their own security measures and adhere to relevant standards based on their specific use cases and organizational requirements.

Standard Used: ISO/IEC 27001

Chapter 4

SYSTEM DESIGN AND METHODOLOGY

4.1 System Architecture

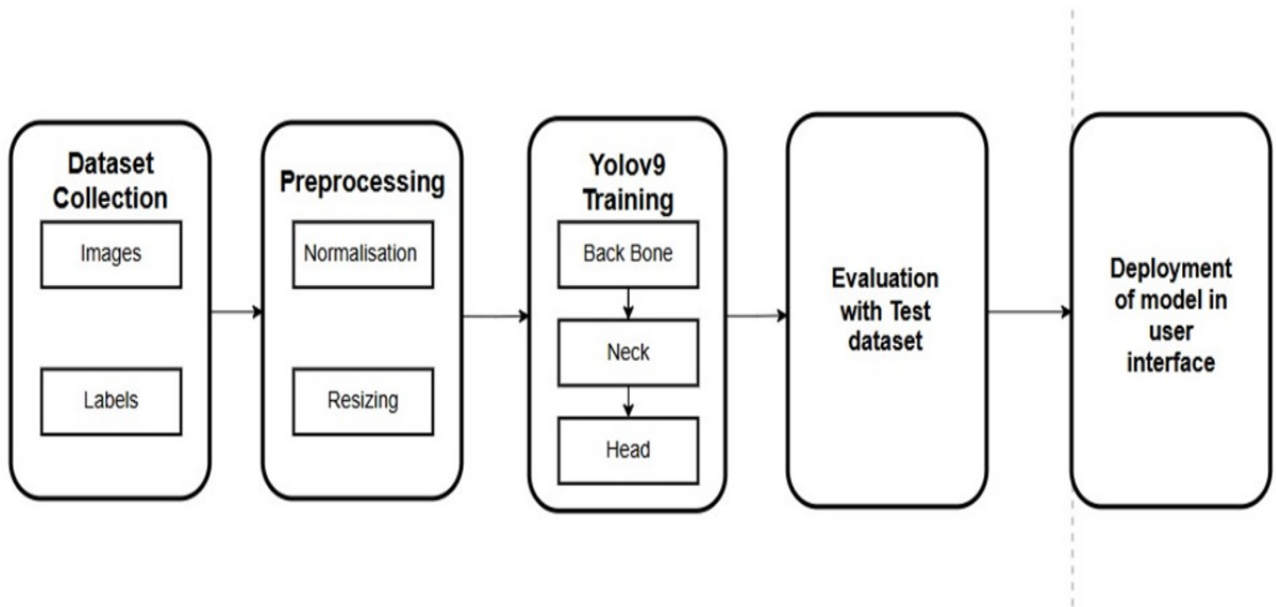


Figure 4.1: Architecture Diagram of Animal Intrusion Detection System

The figure 4.1 illustrates the workflow of the Animal Intrusion Detection system using YOLOv9, beginning with dataset collection, where images and corresponding labels are gathered. This is followed by preprocessing, where the images are resized and normalized to prepare them for training. The preprocessed data is then used in YOLOv9 training, which involves a backbone, neck, and head architecture, and after Evaluation with test dataset, the trained model is finally deployed into a user interface for real-time animal detection.

4.2 Design Phase

4.2.1 Data Flow Diagram

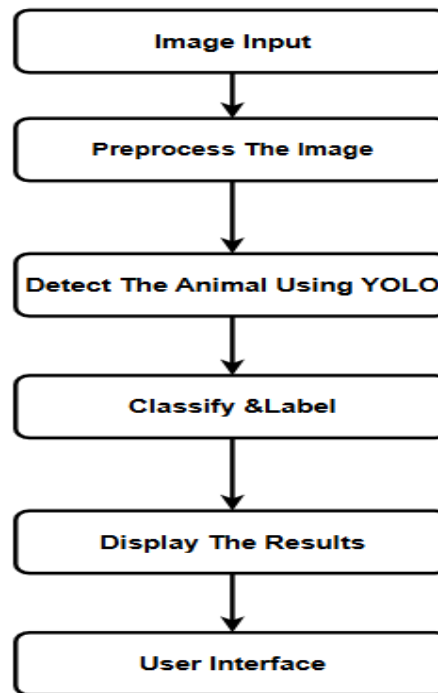


Figure 4.2: Data Flow Diagram

This figure 4.2 depicts a Data Flow Diagram for the animal intrusion detection project using YOLOv9. It illustrates the flow of data through the system, starting from data acquisition via surveillance cameras or sensors, followed by preprocessing and feeding the data into the YOLOv9 model for object detection. The detected outputs (animal types and locations) are then stored in a database or sent to an alert system, ensuring quick response to intrusions. This diagram shows how data is processed and shared between components like data sources, processing units, storage, and end-users.

4.2.2 Sequence Diagram

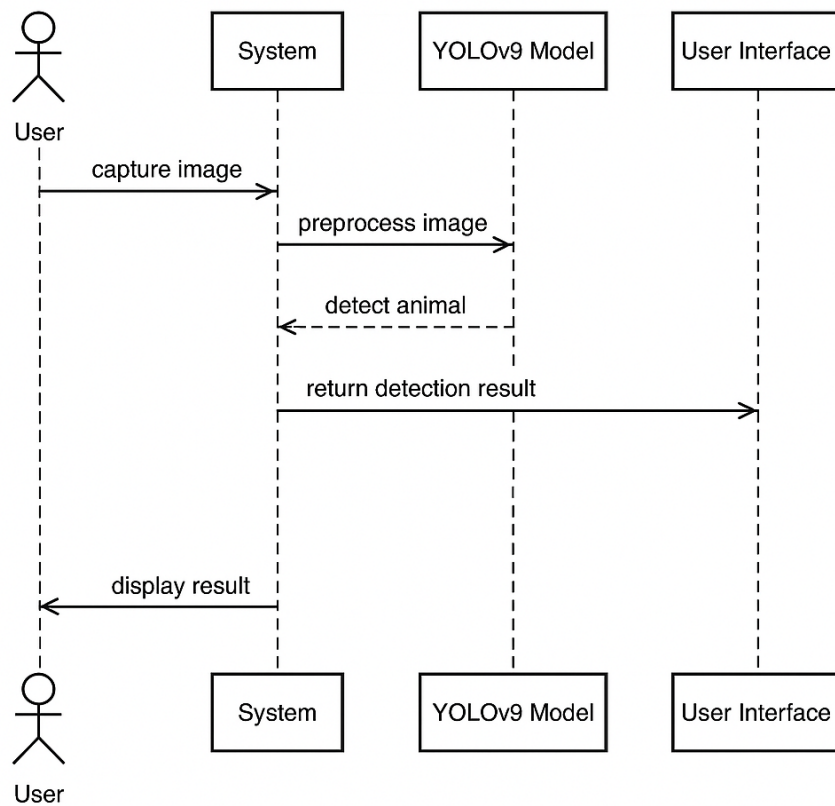


Figure 4.3: Sequence Diagram

This figure 4.3 depicts the sequence diagram for the Animal Intrusion Detection System using YOLOv9 shows how system components interact in order. It starts with the user initiating monitoring, then the camera captures image frames, which are passed to the YOLOv9 model for detecting and classifying animals. The detection results are then displayed to the user. This diagram helps visualize the step-by-step flow of operations for real-time and accurate animal intrusion detection.

4.2.3 Collaboration diagram

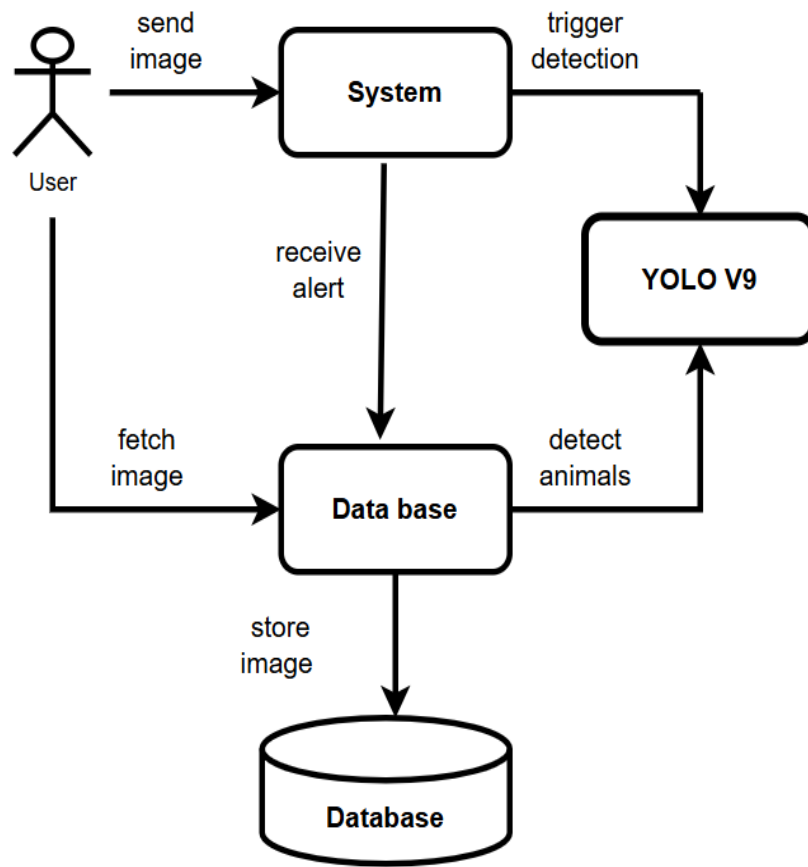


Figure 4.4: Collaboration Diagram

This figure 4.4 depicts the collaboration diagram illustrates how the animal intrusion detection system works. A camera captures images, which are sent to the pre-processing module for formatting. The processed images are passed to the YOLOv9 model, which detects animals using trained data from the dataset. The detection results are displayed through the user interface, and if a threat is detected, an alert system notifies the user.

4.2.4 Activity Diagram

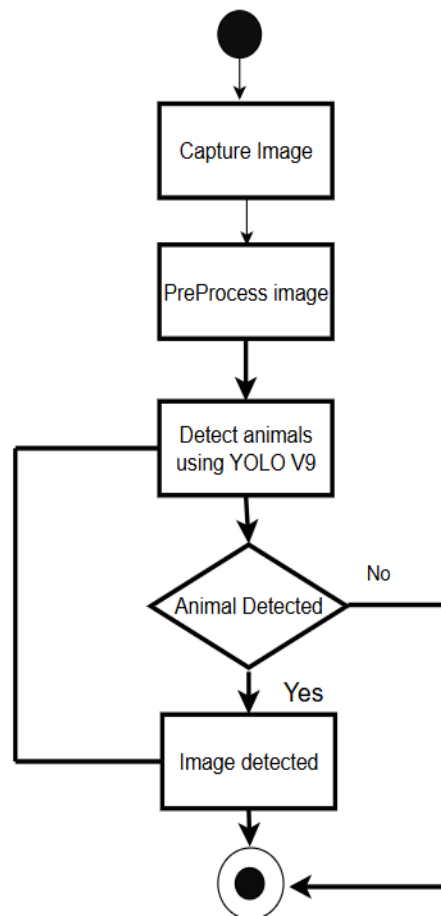


Figure 4.5: Activity Diagram

This figure 4.5 depicts the activity diagram illustrates the step-by-step workflow of the animal intrusion detection system using YOLOv9. It begins with capturing images or video frames, followed by data preprocessing and model inference. Based on detection results, alerts are triggered if an animal is identified, and the output is logged for further analysis or action.

4.3 Algorithm & Pseudo Code

4.3.1 Algorithm

1. Import necessary libraries for data preprocessing, visualization, and model training.
2. Collect and organize a dataset of various animal images with annotated bounding boxes.

3. Preprocess the data by resizing images, normalizing pixel values techniques.
4. Convert all files into YOLOv9-compatible format for training.
5. Split the dataset into training and validation sets for performance evaluation.
6. Configure the YOLOv9 model with appropriate dataset and hyperparameters.
7. Train the YOLOv9 model on the prepared dataset for a fixed number of epochs.
8. Monitor training metrics such as loss, precision, recall, and mAP during training.
9. Evaluate the model using the validation set to check detection performance.
10. Develop a user interface to allow users to upload images for detection.
11. Load the trained model and perform animal detection, displaying results with labels and bounding boxes.

4.3.2 Pseudo Code

```

1 Caption: Animal Intrusion Detection using YOLOv9
2 Input: TRAIN\_SET (train images, train labels), TEST\_SET (test images), CONF\_THRESHOLD
3 Output: Detection results with class labels and bounding boxes
4
5 Begin
6
7 1: Initialize YOLOv9 engine and configuration parameters
8 2: Preprocess dataset images: resize, normalize, remove empty/missing labels
9 3: Load model with pretrained weights or custom checkpoint
10 4: if: training = \texttt{True} then:
11     Train model on TRAIN\_SET (images, labels)
12     Save trained model
13 end if
14
15 5: if: testing = \texttt{True} then:
16     Load trained model
17     For each image in TEST\_SET:
18         Run inference using YOLOv9
19         Apply confidence threshold and non-max suppression
20         Store detection results (bounding box, class label, confidence)
21     end if
22
23 6: Evaluate model using metrics: precision, recall, mAP
24 7: return: detection results and evaluation metrics
25
26 End

```

4.4 Module Description

4.4.1 Collection of Dataset

The dataset used in training the model for animal intrusion detection comprises images of various animals commonly found near farmlands and forest borders. The dataset includes several thousand labeled images featuring different animal species such as elephants, deer, tigers, monkeys, and more, in diverse environmental conditions and backgrounds. Each image is accompanied by annotation files containing class labels and bounding box coordinates specifying the location of animals in the image. These annotations are essential for supervised learning, allowing the object detection model to learn the spatial and class-specific characteristics of each animal. The dataset is structured to ensure that it contains a balanced distribution of images per animal class, improving model generalization and accuracy. The images vary in resolution, angle, lighting, and distance, which introduces the real-world variability the model needs to handle effectively during deployment.

The dataset includes day and night images, crowded scenes, and individual animal instances to ensure robustness across different scenarios. All images are converted to a uniform format and resolution during preprocessing, which ensures compatibility with the YOLOv9 training requirements. To ensure the quality and usability of the data, rigorous preprocessing and verification steps were performed. Duplicate, blurry, and incorrectly labeled images were removed, and missing annotations were corrected or excluded. This careful curation of the dataset ensures that the model is trained on accurate and diverse examples, which is critical for real-time animal detection applications. The dataset utilized for this project is collected from open-source repository and dataset available on platform Kaggle:

<https://www.kaggle.com/code/momiradzemovic/animal-detection-yolov8/input?select=test>

Train Dataset	15 Classes of Animals (1.19 GB)
Test Dataset	15 Classes of Animals (627.5 MB)

Table 4.1: **Dataset Structure**

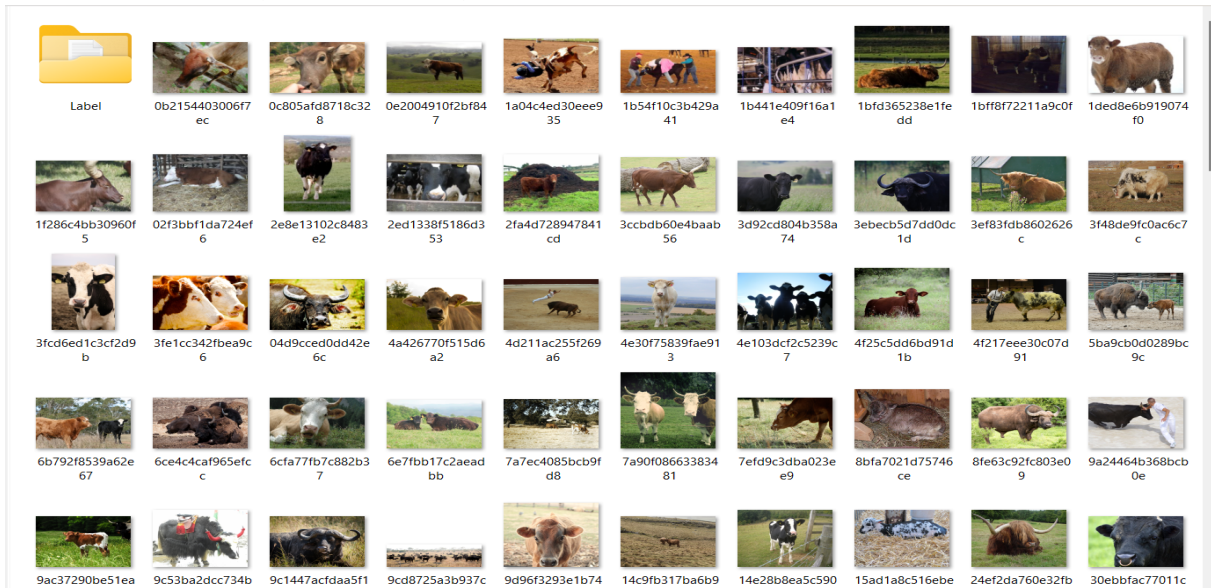


Figure 4.6: Sample of Dataset

4.4.2 Pre-Processing the Dataset

Preprocessing is an essential step in preparing image datasets for deep learning models. It involves a series of transformations that convert raw image data and annotations into a format suitable for training object detection models like YOLOv9. Proper preprocessing improves model efficiency, accuracy, and generalization capabilities. In the context of animal intrusion detection, preprocessing primarily includes resizing images, normalizing pixel values, and removing any missing or empty data samples from the dataset. These steps ensure consistency in input data and reduce computational overhead during model training.

Resizing: Resizing is a fundamental step in preparing input images for YOLOv9. All images are resized to a fixed dimension (e.g., 640×640 pixels) to maintain consistency and compatibility with the model architecture. Resizing also ensures uniform input size across the dataset, which is critical for stable training and efficient GPU utilization. The aspect ratio is typically maintained or adjusted using padding, ensuring that objects are not distorted.

Normalization: Normalization refers to adjusting the bounding box coordinates in the label files to a relative scale based on image dimensions. Each label file contains annotations in the form of class ID, center x, center y, width, and height — all of which must be normalized by dividing by the corresponding image width and

height. This ensures that the bounding box values remain within the range of 0 to 1, making the model resolution-independent. Normalizing label values allows the model to generalize better across different image sizes and improves its ability to detect objects accurately, regardless of image resolution.

Removal of Empty or Missing Data: As part of preprocessing, images without corresponding annotation files or those with empty bounding box data are identified and removed. This step is critical to avoid training the model on irrelevant or unusable data, which can degrade detection performance. Ensuring that every image in the dataset has a valid annotation helps maintain the integrity and quality of the training process.



Figure 4.7: **Pre-Processing the Dataset**

4.4.3 Training the Model

For training the dataset, the YOLOv9 algorithm under deep learning was employed to detect and classify animals in images. YOLOv9 is known for its real-time detection capability and improved accuracy over previous versions. The YOLOv9 architecture introduces enhanced components for improved detection performance and efficiency.

Input Layer: Accepts images in various resolutions and performs initial preprocessing such as normalization and resizing.

Backbone: Responsible for feature extraction from the input image. It utilizes a combination of convolutional layers for capturing spatial features and incorporates a hybrid attention mechanism to improve focus on key regions.

Neck: Enhances feature aggregation for objects at different scales and improves detection of small and distant objects like animals.

Head: Features a dynamic head mechanism that adjusts to varying object sizes, improving detection robustness. It predicts class scores, bounding box coordinates, and confidence scores.

Output Layer: Provides the final detection results with refined bounding boxes, class labels, and confidence scores, filtering out low-confidence detections.

In the context of detecting animal intrusion, YOLOv9 offers several advantages:

Real-Time Detection: YOLOv9 is optimized for speed and is capable of processing multiple frames per second, making it ideal for real-time animal intrusion scenarios where immediate detection is crucial.

High Accuracy in Complex Scenes: With enhanced architectural components and attention mechanisms, YOLOv9 delivers high detection accuracy even in challenging environments with cluttered backgrounds and varying object sizes.

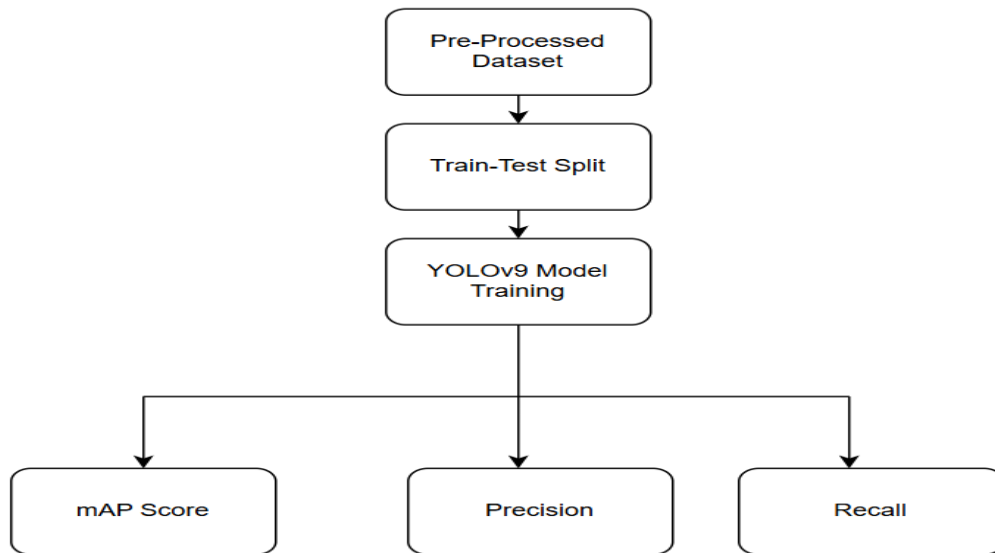


Figure 4.8: Training the Model

4.4.4 Testing the Model

For testing the animal intrusion detection model, a thorough evaluation process is conducted to assess its capability to accurately detect and classify animals in various environments. The model is tested using the test dataset that was not part of the training phase, ensuring an unbiased evaluation of its generalization abilities. Each test image is passed through the trained YOLOv9 model, which processes the image using its learned feature representations to predict bounding boxes, class labels,

and confidence scores. These predictions are then compared with the ground truth annotations provided in the test dataset to determine how accurately the model can identify different animals.

The output of the testing phase includes evaluation metrics specifically used for object detection tasks: Precision, Recall, and Mean Average Precision (mAP). Precision measures the proportion of correct positive predictions among all positive predictions made by the model, indicating its ability to avoid false alarms. Recall evaluates the proportion of actual positives correctly identified by the model, reflecting its sensitivity. mAP provides an overall measure of detection performance by averaging the precision over all recall levels and across all classes. These metrics provide comprehensive insights into the model's effectiveness, ensuring its reliability for real-world deployment in animal intrusion detection scenarios.

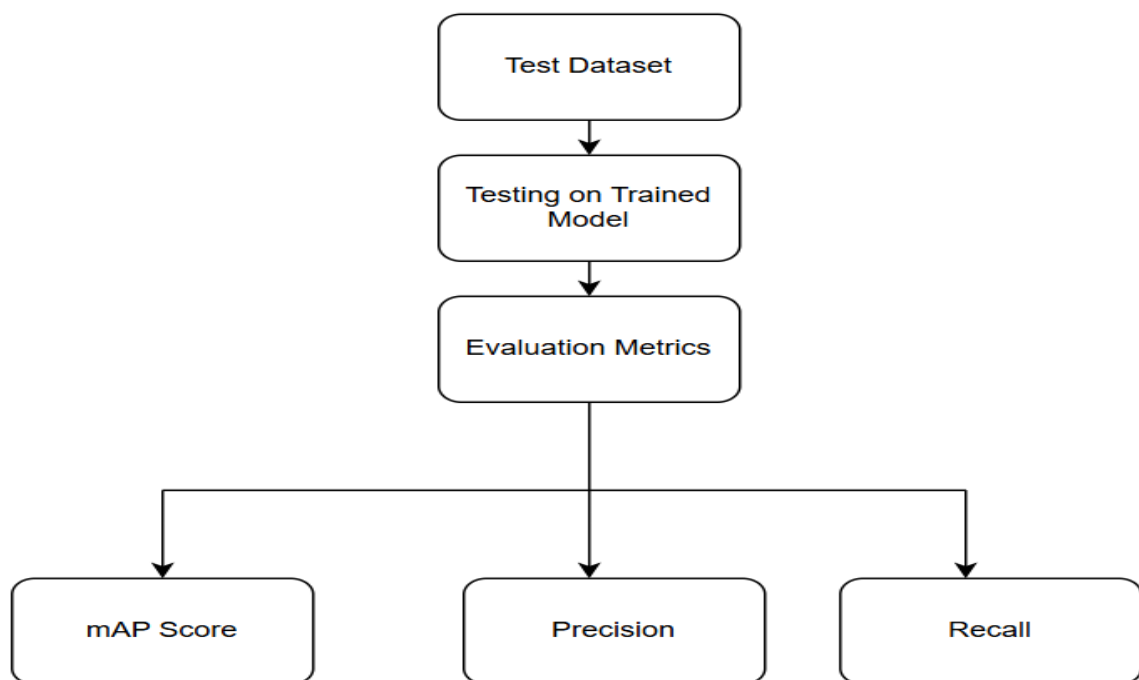


Figure 4.9: Testing the Model

4.5 Steps to execute/run/implement the project

4.5.1 Setup Environment

- Ensure Python is installed.

- Install required packages: PyTorch, OpenCV, NumPy, Matplotlib, and other dependencies.
- Set up Google Colab with GPU support for training.

4.5.2 Download Dataset

- Obtain a labeled dataset of animal images with annotation files in YOLO format.
- Place the dataset in a folder structure compatible with YOLOv9 training.

4.5.3 Data Preprocessing

- Resize and normalize images to match YOLOv9 input requirements.
- Convert annotation files to YOLO format if not already available.

4.5.4 Model Configuration

- Configure the YOLOv9 model settings including image size, batch size, epochs, number of classes, and model architecture.
- Create or edit the dataset YAML file with training/validation paths and class names.

4.5.5 Model Training

- Train the YOLOv9 model using the training dataset in Google Colab.
- Monitor loss, precision, recall, and mAP metrics during training.

4.5.6 Model Evaluation

- Evaluate the trained model using the validation or test dataset.
- Analyze performance metrics such as precision, recall and mAP.

4.5.7 Results and Analysis

- Visualize detection outputs with bounding boxes and labels.
- Analyze strengths and weaknesses of the model using confusion matrix and class-wise metrics.

4.5.8 Model Deployment

- Save the trained model weights for future use.
- Develop a simple user interface using Flask to upload images and display animal detection results.

Chapter 5

IMPLEMENTATION AND TESTING

5.1 Input and Output

5.1.1 Input Design

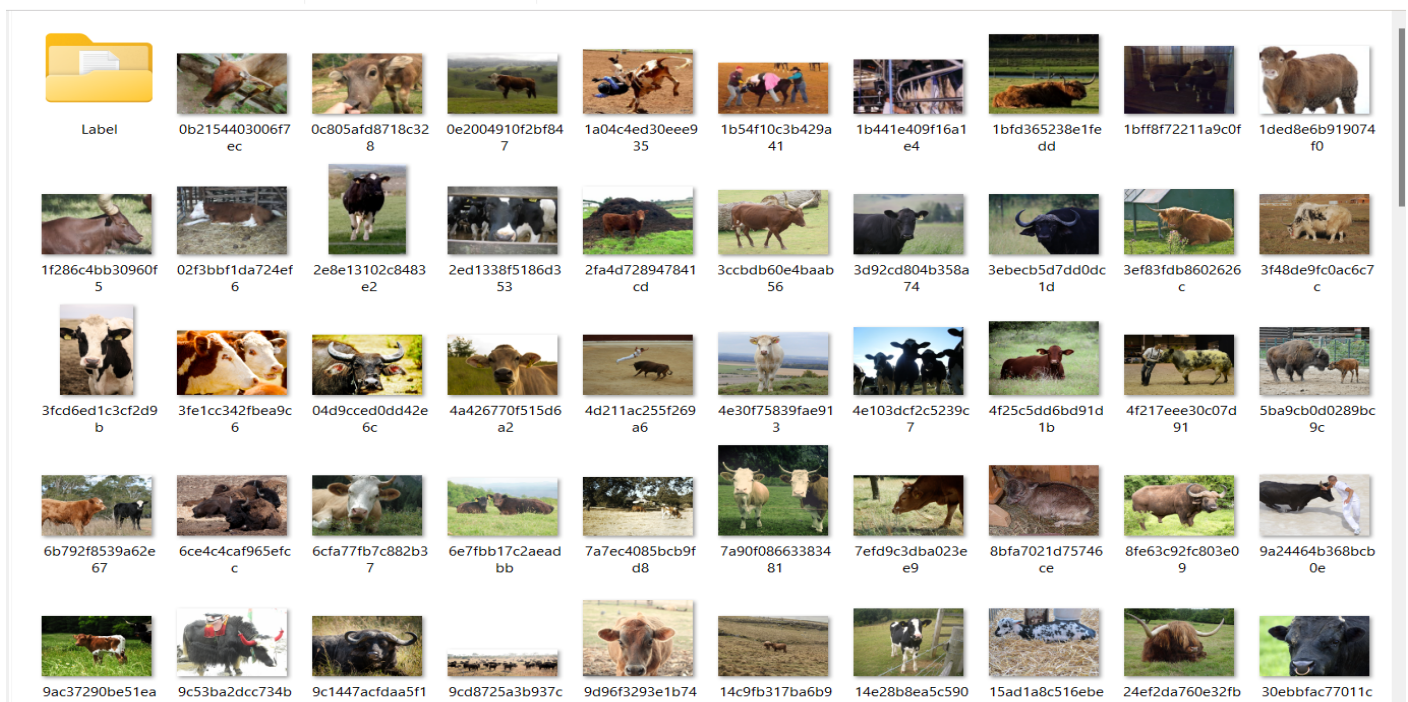


Figure 5.1: Animal Dataset

The figure 5.1 illustrates the input of the system, which comprises a diverse dataset containing annotated images of various animals captured in different environments. This dataset includes visual data representing different species, lighting conditions, backgrounds, and animal positions, providing comprehensive information necessary for accurate animal detection. Each image is paired with corresponding label files that define the bounding boxes and class labels for the animals present, enabling the model to learn meaningful patterns for precise intrusion detection.

5.1.2 Output Design

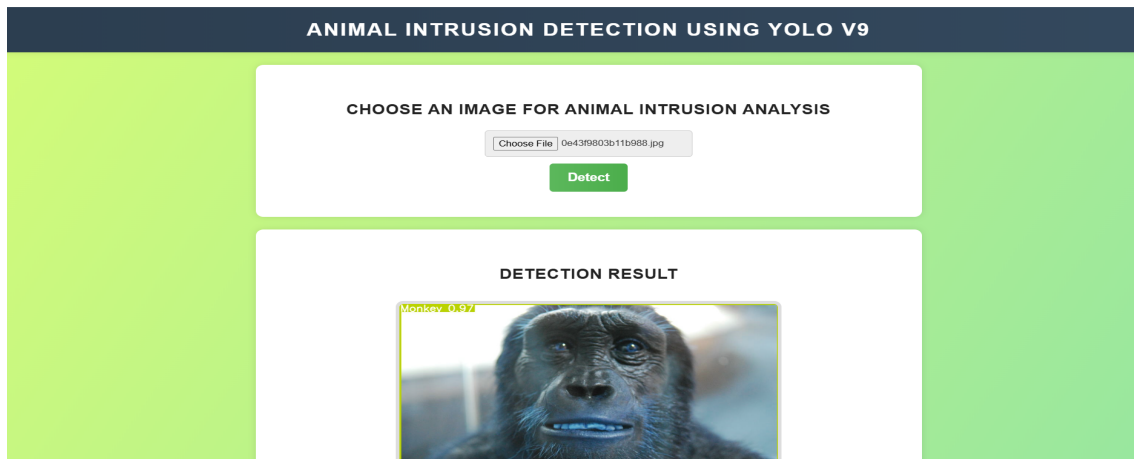


Figure 5.2: Detection of Animals on User Interface

The figure 5.2 displays the evaluation of the model for animal intrusion detection. By analyzing visual features extracted from input images, the model identifies and classifies animals accurately within the scene, enabling effective monitoring and enhancing situational awareness for security and wildlife management.

5.2 Testing

The program undergoes rigorous testing to ensure its accuracy, reliability, and functionality. Various testing methodologies, including unit testing, assess individual components, while integration testing evaluates interactions between these components. Specialized testing for preprocessing techniques and model validates their effectiveness. Additionally, robustness testing examines the program's resilience to adverse conditions. Continuous testing and validation are integral to ensuring the program's efficacy in detecting animals, while maintaining high standards of precision and performance.

5.2.1 Testing Strategies

Unit Testing

Unit testing is a software testing approach that involves the evaluation of individual units or components within a software application in isolation. These units typically refer to the smallest testable parts of the code, such as functions, meth-

ods, or procedures. The primary objective of unit testing is to confirm that each unit behaves as intended, independently of the rest of the application.

Input

```
1 import os
2
3 # Define dataset path
4 dataset_path = "/content/YOLO_dataset"
5
6 # Iterate through train and test sets
7 for split in ["Train", "Test"]:
8     split_path = os.path.join(dataset_path, split)
9
10    print(f"\n    Checking {split} dataset...\n")
11
12    # Iterate over all animal categories
13    for category in os.listdir(split_path):
14        category_path = os.path.join(split_path, category)
15
16        if not os.path.isdir(category_path):
17            continue # Skip if not a directory (e.g., files like train.zip)
18
19        image_folder = category_path
20        label_folder = os.path.join(category_path, "Label")
21
22        # Get list of image and label filenames (without extensions)
23        image_files = {os.path.splitext(f)[0] for f in os.listdir(image_folder) if f.endswith((''.jpg', '.png'))}
24        label_files = {os.path.splitext(f)[0] for f in os.listdir(label_folder) if f.endswith('.txt')}
25
26        # Find missing labels & images
27        missing_labels = image_files - label_files # Images without labels
28        missing_images = label_files - image_files # Labels without images
29
30        # Print results per category
31        if not missing_labels and not missing_images:
32            print(f"    {category}: All images have corresponding label files!")
33        else:
34            if missing_labels:
35                print(f"    {category}: {len(missing_labels)} images are missing labels:")
36                print(list(missing_labels)[:5], "...") # Show only first 5 missing items
37            if missing_images:
38                print(f"    {category}: {len(missing_images)} labels are missing images:")
39                print(list(missing_images)[:5], "...") # Show only first 5 missing items
```

Test result

```
# Get list of image and label filenames (without extensions)
image_files = {os.path.splitext(f)[0] for f in os.listdir(image_folder) if f.endswith((' .jpg', ' .png'))}
label_files = {os.path.splitext(f)[0] for f in os.listdir(label_folder) if f.endswith(' .txt')}

# Find missing labels & images
missing_labels = image_files - label_files # Images without labels
missing_images = label_files - image_files # Labels without images

# Print results per category
if not missing_labels and not missing_images:
    print(f"✅ {category}: All images have corresponding label files!")
else:
    if missing_labels:
        print(f"⚠️ {category}: {len(missing_labels)} images are missing labels:")
        print(list(missing_labels)[:5], "...") # Show only first 5 missing items
    if missing_images:
        print(f"⚠️ {category}: {len(missing_images)} labels are missing images:")
        print(list(missing_images)[:5], "...") # Show only first 5 missing items
```

🔍 Checking Train dataset...

- ✅ Bear: All images have corresponding label files!
- ✅ Sheep: All images have corresponding label files!
- ✅ Snake: All images have corresponding label files!
- ✅ Bull: All images have corresponding label files!
- ✅ Elephant: All images have corresponding label files!
- ✅ Rabbit: All images have corresponding label files!
- ✅ Cattle: All images have corresponding label files!
- ✅ Goat: All images have corresponding label files!
- ✅ Deer: All images have corresponding label files!
- ✅ Cheetah: All images have corresponding label files!
- ✅ Tiger: All images have corresponding label files!
- ✅ Lion: All images have corresponding label files!
- ✅ Pig: All images have corresponding label files!
- ✅ Monkey: All images have corresponding label files!
- ✅ Horse: All images have corresponding label files!

🔍 Checking Test dataset...

- ✅ Bear: All images have corresponding label files!

Figure 5.3: Result of Unit Test

Integration Testing

Integration testing is a software testing methodology that focuses on evaluating the interactions and dependencies between different components within a larger system. The goal of integration testing is to ensure that individual components, when combined, function correctly as a cohesive unit. This type of testing helps uncover errors related to data flow, control flow, and communication between modules.

Input

```
1 %cd {HOME}
2
3 !python val.py \
4 --img 640 --batch 32 --conf 0.001 --iou 0.7 --device 0 \
5 --data /content/Formatted_dataset/dataset.yaml \
6 --weights {HOME}/runs/train/exp2/weights/best.pt
```

Test result

```
%cd {HOME}
!python val.py \
--img 640 --batch 32 --conf 0.001 --iou 0.7 --device 0 \
--data /content/Formatted_dataset/dataset.yaml \
--weights {HOME}/runs/train/exp2/weights/best.pt

/content/yolov9
val: data=/content/Formatted_dataset/dataset.yaml, weights=['/content/yolov9/runs/train/exp2/weights/best.pt'], batch_size=32, imgsz=640, conf_thres=0.001, iou_thres=0.7, max_det=300, task=YOLOv5 1e33dbb Python-3.11.11 torch-2.5.0+cu124 CUDA:0 (Tesla T4, 15095MiB)

/content/yolov9/models/experimental.py:75: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. ckpt = torch.load(attempt_download(w), map_location='cpu') # load
Fusing layers...
gelan-c summary: 467 layers, 25422525 parameters, 0 gradients, 102.5 GFLOPs
val: Scanning /content/Formatted_dataset/Test/labels.cache... 1644 images, 0 backgrounds, 0 corrupt: 100% 1644/1644 [00:00<?, ?it/s]
      Class   Images  Instances    P      R   mAP50   mAP50-95:   2% 1/52 [00:02<02:12, 2.59%/it]Exception in thread Thread-3 (plot_images):
Traceback (most recent call last):
  File "/usr/lib/python3.11/threading.py", line 1045, in _bootstrap_inner
    self.run()
  File "/usr/lib/python3.11/threading.py", line 982, in run
    self._target(*self._args, **self._kwargs)
  File "/content/yolov9/utils/plots.py", line 300, in plot_images
    annotator.box_label(box, label, color=color)
  File "/content/yolov9/utils/plots.py", line 86, in box_label
    w, h = self.font.getsize(label) # text width, height
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'FreeTypeFont' object has no attribute 'getsize'
Exception in thread Thread-4 (plot_images):
Traceback (most recent call last):
  File "/usr/lib/python3.11/threading.py", line 1045, in _bootstrap_inner
```

Figure 5.4: Result of Integration Test

Functional Testing

Functional testing is a crucial phase in software testing that focuses on verifying whether a software application or system performs its intended functions as specified in the requirements. It aims to ensure that the software meets the functional expectations and behaves according to the design specifications. During functional testing, the application is tested against its functional requirements, and test cases are designed to cover various scenarios, inputs, and system states.

Input

```
1
2 import os
3 import shutil
4
5 # Define original dataset paths
6 original_dataset_path = "/content/YOLO_dataset"
7 train_path = os.path.join(original_dataset_path, "Train")
8 test_path = os.path.join(original_dataset_path, "Test")
9
10 # Define a new folder for the copied dataset
11 new_dataset_path = "/content/Formatted_dataset"
12 new_train_path = os.path.join(new_dataset_path, "Train")
13 new_test_path = os.path.join(new_dataset_path, "Test")
14
15 # Ensure new dataset folders exist
16 for path in [new_train_path, new_test_path]:
17     os.makedirs(os.path.join(path, "images"), exist_ok=True)
18     os.makedirs(os.path.join(path, "labels"), exist_ok=True)
19
20 # Function to reorganize images and labels
21 def organize_and_copy_data(src_base_path, dest_base_path):
22     for class_folder in os.listdir(src_base_path): # Iterate through class subfolders
23         class_folder_path = os.path.join(src_base_path, class_folder)
24
25         if os.path.isdir(class_folder_path): # Ensure it's a folder
26             for item in os.listdir(class_folder_path):
27                 item_path = os.path.join(class_folder_path, item)
28
29                 if item.endswith((".jpg", ".png", ".jpeg")): # Copy images
30                     shutil.copy(item_path, os.path.join(dest_base_path, "images", item))
31
32                 elif os.path.isdir(item_path) and item == "Label": # Handle label subfolder
33                     for label_file in os.listdir(item_path):
34                         if label_file.endswith(".txt"):
35                             shutil.copy(os.path.join(item_path, label_file), os.path.join(
36                                 dest_base_path, "labels", label_file))
37
38 # Run for Train and Test datasets
39 organize_and_copy_data(train_path, new_train_path)
40 organize_and_copy_data(test_path, new_test_path)
41
42 print("    Images and labels copied successfully to Formatted_dataset!")
```

Test result

```
# Function to reorganize images and labels
def organize_and_copy_data(src_base_path, dest_base_path):
    for class_folder in os.listdir(src_base_path): # Iterate through class subfolders
        class_folder_path = os.path.join(src_base_path, class_folder)

        if os.path.isdir(class_folder_path): # Ensure it's a folder
            for item in os.listdir(class_folder_path):
                item_path = os.path.join(class_folder_path, item)

                if item.endswith((".jpg", ".png", ".jpeg")): # Copy images
                    shutil.copy(item_path, os.path.join(dest_base_path, "images", item))

                elif os.path.isdir(item_path) and item == "Label": # Handle label subfolder
                    for label_file in os.listdir(item_path):
                        if label_file.endswith(".txt"):
                            shutil.copy(os.path.join(item_path, label_file), os.path.join(dest_base_path, "labels", label_file))

# Run for Train and Test datasets
organize_and_copy_data(train_path, new_train_path)
organize_and_copy_data(test_path, new_test_path)

print("✅ Images and labels copied successfully to Formatted_dataset!")
```

➡️ ✅ Images and labels copied successfully to Formatted_dataset!

Figure 5.5: Result of Functional Test

5.2.2 Performance Evaluation

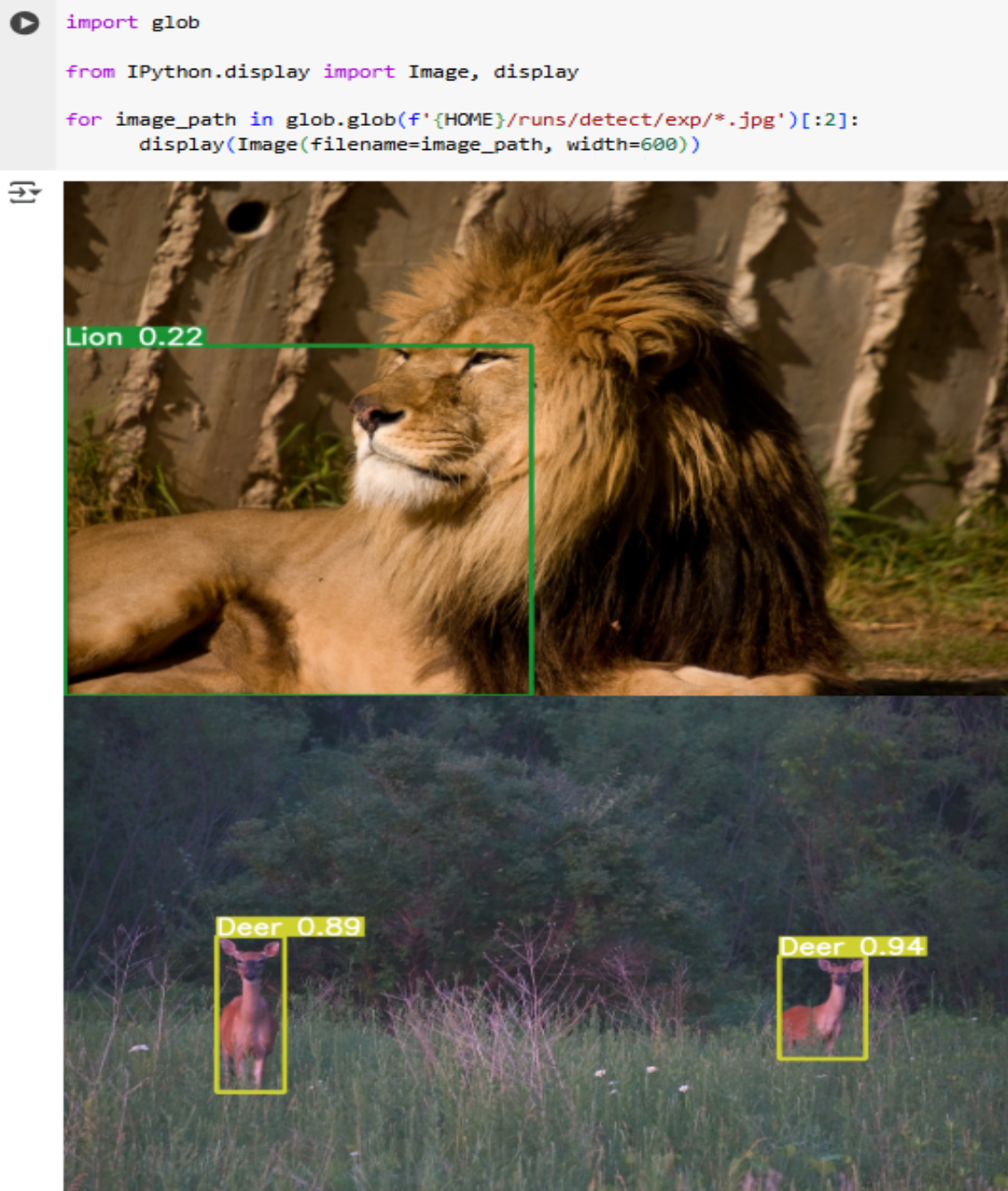


Figure 5.6: Test Image

Chapter 6

RESULTS AND DISCUSSIONS

6.1 Efficiency of the Proposed System

The proposed animal intrusion detection system demonstrates high efficiency in identifying and classifying animals in real-time surveillance environments. By employing the YOLOv9 architecture, the system achieves superior detection accuracy due to its advanced components. These enhancements enable the system to detect even small or distant animals effectively, which is particularly critical for early warning in farmlands and forest borders. The use of precise bounding boxes and confidence scores improves localization and reduces false positives, thereby making the system highly reliable in various environmental conditions. The training phase leverages a robust dataset with annotated images, allowing the model to learn and generalize well to new scenarios.

The efficiency of the system is also evident in its real-time detection capability, which is essential for practical deployment in surveillance applications. The inference speed of YOLOv9, combined with optimized image preprocessing steps like normalization, resizing ensures minimal latency in detection. The model's lightweight structure allows it to be integrated into edge devices like Raspberry Pi or NVIDIA Jetson boards for real-time operations without needing high-end GPUs. Additionally, the modular design of the system supports seamless integration with alert systems, automated deterrents, or cloud-based dashboards. These features contribute to the overall efficiency and practicality of the proposed solution, enabling farmers and forest authorities to act swiftly and prevent potential damage caused by wild animal intrusions.

6.2 Comparison of Existing and Proposed System

Existing system: (Manual Analysis)

The existing system for detecting animal intrusions in agricultural or forest border areas primarily relies on manual surveillance methods, such as watchtowers, patrolling personnel, or basic motion sensor-based alarms. These approaches are labor-intensive, time-consuming, and prone to human error, often resulting in delayed responses or false alarms. Manual analysis also lacks consistency in monitoring during night-time or adverse weather conditions, reducing the overall reliability of intrusion detection. Furthermore, these traditional methods do not provide a way to identify the specific type of animal or assess the level of threat, which limits the scope for appropriate response mechanisms. As a result, the manual system proves to be inefficient, especially for large-scale or remote agricultural lands, leading to crop damage, financial loss, and safety concerns. The need for automation and intelligent monitoring systems has become crucial to overcome the shortcomings of manual surveillance.

Proposed system: (YOLO V9)

The proposed system leverages the advanced capabilities of the YOLOv9 algorithm for real-time animal intrusion detection. This deep learning-based approach enables the system to automatically detect and classify various animals in surveillance footage with high accuracy and speed. YOLOv9 introduces architectural improvements such as hybrid attention mechanisms and dynamic head structures, which enhance the model's ability to focus on key regions and detect objects of varying sizes, even in complex or cluttered environments. Unlike traditional systems, this model processes the entire image in a single forward pass, making it highly efficient for real-time applications. The integration of YOLOv9 significantly reduces the need for manual intervention and minimizes false alarms by providing precise bounding boxes and confidence scores for detected animals. This not only ensures timely alerts but also supports decision-making processes by identifying the specific type of intruder, thereby enhancing the security and protection of agricultural or forested zones.

6.3 Comparative Analysis-Table

Model Name	Mean Average Precision (mAP)	Recall	Precision
YOLO V9 (Proposed)	0.964	0.981	0.986
Cascaded YOLO V8 (Johnwesily)	0.923	0.937	0.942
YOLO V8(Kathir)	0.932	0.945	0.952
YOLO V5(Praveen)	0.917	0.932	0.961

Table 6.1: Comparison of Different Models with Proposed Model

The table 6.1 presents a comparative evaluation of various YOLO model versions based on three standard performance metrics: Mean Average Precision (mAP), Recall, and Precision. The proposed YOLOv9 model achieves the highest mAP of 0.964, Recall of 0.981, and Precision of 0.986, indicating its superior performance in accurately detecting animal intrusions with minimal false positives and false negatives.

6.4 Comparative Analysis-Graphical Representation and Discussion

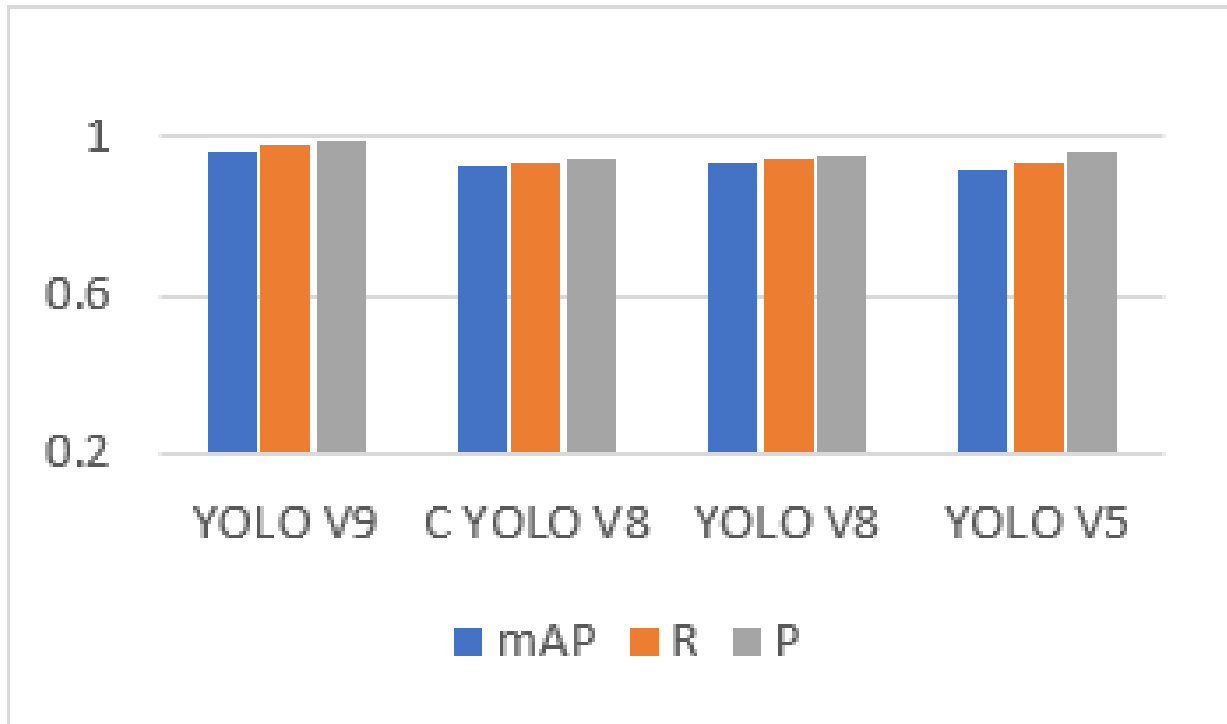


Figure 6.1: Comparison Graph

The figure 6.1 illustrates the performance of various YOLO versions—YOLOv9, CYOLOv8, YOLOv8, and YOLOv5—based on three crucial evaluation metrics: mean Average Precision (mAP), Recall (R), and Precision (P). Among all the versions, YOLOv9 consistently outperforms others across all metrics, achieving near-perfect scores, indicating its superior detection accuracy and robustness.

Comparatively, CYOLOv8 and YOLOv8 also deliver strong performance but slightly lag behind YOLOv9, particularly in recall and precision, suggesting minor deficiencies in detecting all true positives or filtering false positives effectively. YOLOv5 shows the lowest performance in all three metrics, highlighting its limitations in handling complex detection scenarios like animal intrusion. Overall, the graph clearly demonstrates YOLOv9's efficiency in terms of detection precision, robustness, and generalization, making it the most suitable model for real-time animal intrusion detection tasks.

Chapter 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 Summary

This project focuses on developing an intelligent Animal Intrusion Detection System using the YOLOv9 object detection algorithm. The system aims to identify and localize animals entering restricted zones, particularly agricultural fields or forest-protected areas. By leveraging a custom-labeled dataset of animal images and applying preprocessing techniques such as image resizing, label normalization the model is trained to detect multiple classes of animals with high precision. The model architecture incorporates improved components like hybrid attention mechanisms and dynamic heads, enhancing the ability to detect small or distant animals effectively. Training and testing processes were executed with appropriate evaluation metrics including precision, recall, and mean Average Precision (mAP), ensuring the reliability and accuracy of the system.

The proposed system significantly improves over traditional manual monitoring techniques by automating the detection process and reducing response time. The use of deep learning enables scalability, allowing the model to adapt to new animal classes and environmental conditions with additional training. Furthermore, by integrating this system into a real-time alert mechanism, it becomes a powerful tool for farmers, forest officials, and wildlife conservationists. The outcome of this project demonstrates how AI-powered surveillance can transform conventional security systems and contribute to the prevention of crop damage, human-wildlife conflicts, and unauthorized intrusions. The efficiency and robustness of the proposed solution mark a significant step forward in the field of automated monitoring and smart agriculture.

7.2 Limitations

Despite the promising results achieved by the proposed YOLOv9-based Animal Intrusion Detection System, certain limitations exist that may affect its deployment and scalability. One of the primary limitations is the dependence on the quality and diversity of the dataset. If the dataset lacks sufficient variation in lighting conditions, angles, or animal postures, the model's ability to generalize to real-world scenarios can be significantly reduced. Additionally, the system may struggle to detect animals in highly cluttered environments, poor visibility, or under extreme weather conditions. The presence of similar-looking objects or partial occlusions can also lead to false detections or missed intrusions.

Furthermore, the current implementation is designed for detection but does not incorporate real-time tracking or behavioral analysis of the detected animals. It relies heavily on powerful GPUs for training and testing, which might not be readily available in rural or field-deployed environments. Integration with edge devices requires further optimization to reduce model size and computation. Moreover, the system lacks multilingual or multimodal alert systems for diverse user bases and does not currently handle non-animal intrusions.

7.3 Future Enhancements

The current animal intrusion detection system based on YOLOv9 provides a solid foundation for real-time monitoring, but there is significant scope for future enhancements. One potential improvement is the integration of real-time tracking algorithms along with detection. By incorporating object tracking models such as Deep SORT or ByteTrack, the system could not only detect intruding animals but also monitor their movement paths and behaviors over time. This would allow for more informed decisions and better responses from security or farm management personnel. Another enhancement could include the use of infrared or thermal imaging to support detection under low light or nighttime conditions, expanding usability around the clock.

Additionally, efforts can be directed toward optimizing the model for deployment on low-power edge devices such as Raspberry Pi or NVIDIA Jetson Nano. This

would enable real-time detection directly at remote locations without needing constant internet access or cloud support. Incorporating AI-driven alert systems via SMS, audio alarms, or multilingual notifications can also make the system more user-friendly and inclusive. Furthermore, a user interface can be developed to display detections, logs, and analytics in a centralized dashboard, making the system more interactive and valuable for end-users. These future enhancements will contribute to making the solution more robust, accessible, and suitable for large-scale deployment in various environmental settings.

Chapter 8

SUSTAINABLE DEVELOPMENT GOALS (SDGs)

8.1 Alignment with SDGs

The Animal Intrusion Detection System using YOLOv9 aligns with several United Nations Sustainable Development Goals (SDGs), contributing to a more secure, sustainable, and technologically empowered society. The project utilizes cutting-edge deep learning techniques to prevent animal intrusions in agricultural fields or urban boundaries, thereby enhancing safety, reducing crop damage, and fostering co-existence between human habitats and wildlife.

The project contributes to the following SDGs:

- **SDG 9: Industry, Innovation, and Infrastructure** – By implementing advanced artificial intelligence techniques and computer vision for real-world security and agricultural challenges, the project promotes sustainable technological innovation and supports the development of smart surveillance infrastructure.
- **SDG 15: Life on Land** – The system supports conservation efforts by reducing harmful human-animal conflicts. Early detection and alert systems can prevent dangerous situations, allowing wildlife to be monitored and safely redirected without harm.
- **SDG 11: Sustainable Cities and Communities** – By enhancing safety in sub-

urban and rural areas where human-wildlife interaction is common, the system contributes to building inclusive, resilient, and safe communities.

- **SDG 2: Zero Hunger** – By mitigating the destruction of crops caused by intruding animals, the system helps farmers protect food resources and maintain agricultural productivity, supporting food security initiatives.

8.2 Relevance of the Project to Specific SDG

The project is directly aligned with SDG 15: Life on Land, as it promotes the coexistence of human settlements and wildlife by minimizing conflicts through a non-invasive animal intrusion detection system. By detecting animals entering agricultural or residential zones in real-time, the system helps reduce instances of human-wildlife conflict, protecting both farmers and wild animals. This aligns with global efforts to conserve biodiversity and maintain ecological balance while ensuring sustainable use of terrestrial ecosystems.

In addition, the project strongly supports SDG 9: Industry, Innovation, and Infrastructure through the integration of advanced deep learning techniques like YOLOv9 and real-time video analytics. The innovative nature of this system enhances rural infrastructure by enabling intelligent surveillance in remote or low-resource settings using efficient and scalable models. The use of edge computing and potential IoT deployment also underscores its capacity for wide-scale implementation without heavy energy consumption, thereby fostering innovation while promoting sustainable and inclusive infrastructure growth.

8.3 Potential Social and Environmental Impact

Social Impact: The proposed Animal Intrusion Detection System offers significant social benefits, especially for rural and agricultural communities where crop damage due to wild animals is a recurring issue. By providing a real-time monitoring and alert system, it empowers farmers to safeguard their livelihood, reduces economic losses, and minimizes human-wildlife conflict. The system also enhances safety for local residents, especially in areas near forest fringes. The affordability and simplicity of the system make it accessible for underserved and low-resource communities, promoting digital inclusion and equitable access to modern technological solutions.

Environmental Impact: From an environmental standpoint, the system contributes to sustainable living and ecological balance. By detecting intrusions early, it allows for non-violent intervention methods that can prevent harm to wildlife. This encourages humane and conservation-friendly practices. Additionally, the model is optimized for use on low-power computing devices, reducing energy consumption and enabling deployment in remote areas without heavy computational infrastructure. Such efficiency supports resource optimization and contributes to lowering the overall carbon footprint of the system.

8.4 Economic Feasibility

The Animal Intrusion Detection system using YOLOv9 was developed at minimal cost using available resources. All resources utilized, including the computer GPU for training, open-source libraries, and publicly available datasets, were freely accessible. No expenses were incurred for hardware, software. This demonstrates that the system is not only technically effective but also highly economical, making it a practical solution for real-world deployment without financial barriers.

Chapter 9

PLAGIARISM REPORT

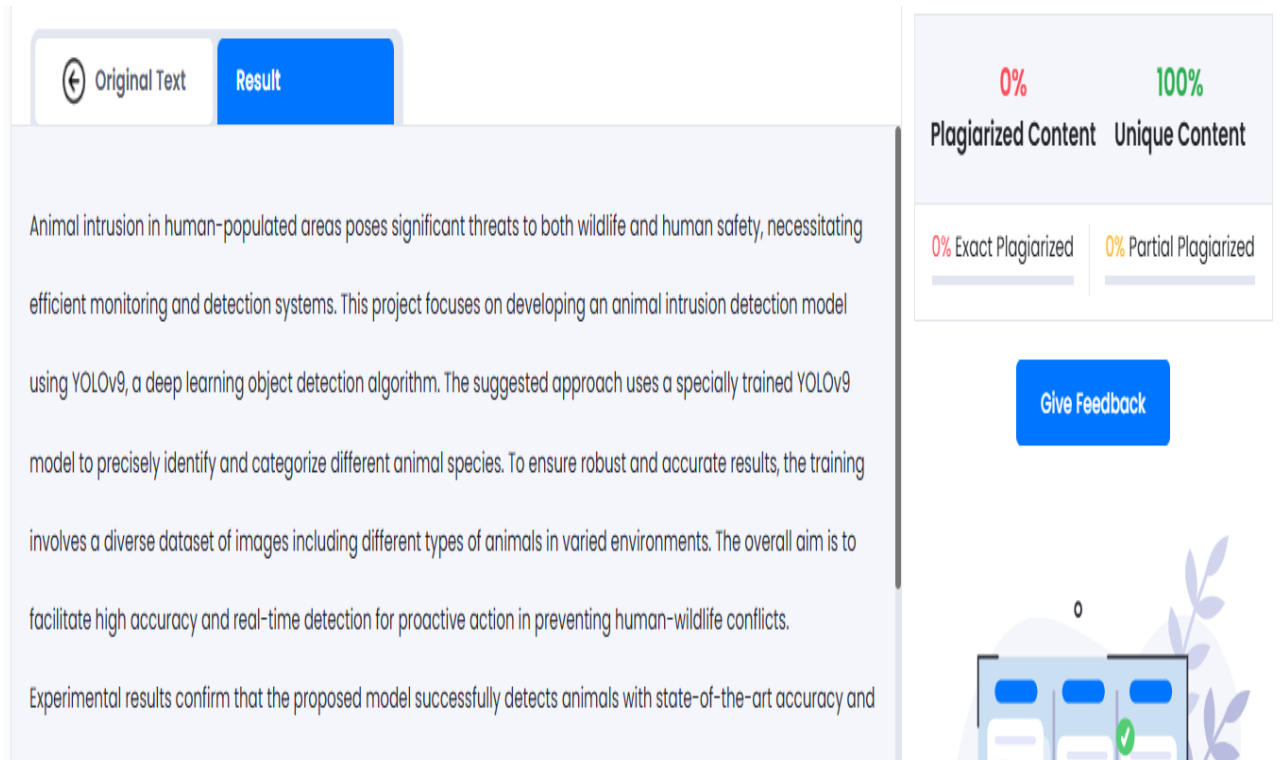


Figure 9.1: **Plagiarism**

Chapter 10

SOURCE CODE

10.1 Source Code

```
1 !pip install torch==2.5.0 torchvision torchaudio
2 !python -c "import torch; print(torch.__version__)"
3 import shutil
4 import os
5
6 # Define paths
7 drive_dataset_path = "/content/drive/MyDrive/YOLO dataset"
8 colab_dataset_path = "/content/YOLO_dataset"
9
10 # Ensure the Colab dataset folder exists
11 os.makedirs(colab_dataset_path, exist_ok=True)
12
13 # Define zip filenames
14 zip_files = ["Train.zip", "Test.zip"]
15
16 # Copy zip files from YOLO dataset folder
17 for zip_file in zip_files:
18     src = os.path.join(drive_dataset_path, zip_file)
19     dest = os.path.join(colab_dataset_path, zip_file)
20
21     if os.path.exists(src):
22         print(f"      Copying {zip_file} to Colab (this may take a while)...")
23         shutil.copy(src, dest)
24         print(f"      {zip_file} copied successfully!")
25     else:
26         print(f"      {zip_file} not found in YOLO dataset folder!")
27     import zipfile
28
29 colab_dataset_path = "/content/YOLO_dataset"
30
31 # Extract train.zip and test.zip
32 for zip_file in ['Train.zip', 'Test.zip']:
33     zip_path = os.path.join(colab_dataset_path, zip_file)
34     extract_path = os.path.join(colab_dataset_path, zip_file.replace('.zip', ''))
35
36     if os.path.exists(zip_path):
37         try:
38             with zipfile.ZipFile(zip_path, 'r') as zip_ref:
```

```

39         # Check for corruption
40         if zip_ref.testzip() is not None:
41             print(f"    {zip_file} is corrupted! Skipping extraction.")
42             continue
43
44         # Extract only if not already extracted
45         if not os.path.exists(extract_path):
46             print(f"    Extracting {zip_file} (this may take a while)...")
47             zip_ref.extractall(extract_path)
48             print(f"    {zip_file} extracted successfully!")
49         else:
50             print(f"    {zip_file} is already extracted!")
51
52     except zipfile.BadZipFile:
53         print(f"    ERROR: {zip_file} is not a valid ZIP file! Skipping extraction.")
54
55     else:
56         print(f"    {zip_file} not found in Colab dataset folder!")
57 import os
58 import cv2
59 import matplotlib.pyplot as plt
60
61 # Define dataset paths
62 dataset_path = "/content/Formatted_dataset"
63 subsets = ["Train", "Test"] # Modify if you have different subfolder names
64
65 # Function to visualize dataset
66 def visualize_dataset(subset="Train", num_images=5):
67     images_path = os.path.join(dataset_path, subset, "images")
68     labels_path = os.path.join(dataset_path, subset, "labels")
69
70     image_files = [f for f in os.listdir(images_path) if f.endswith((' .jpg', ' .png'))]
71
72     for img_file in image_files[:num_images]:
73         img_path = os.path.join(images_path, img_file)
74         label_file = os.path.join(labels_path, os.path.splitext(img_file)[0] + ".txt")
75
76         # Load image
77         img = cv2.imread(img_path)
78         img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
79
80         if os.path.exists(label_file):
81             with open(label_file, "r") as f:
82                 for line in f.readlines():
83                     data = line.strip().split()
84                     class_id = int(data[0])
85                     x_center, y_center, width, height = map(float, data[1:])
86
87                     # Convert YOLO format to pixel coordinates
88                     h, w, _ = img.shape

```

```

89         x1 = int((x_center - width / 2) * w)
90         y1 = int((y_center - height / 2) * h)
91         x2 = int((x_center + width / 2) * w)
92         y2 = int((y_center + height / 2) * h)
93
94         # Draw bounding box
95         cv2.rectangle(img, (x1, y1), (x2, y2), (255, 0, 0), 2)
96         cv2.putText(img, f"Class {class_id}", (x1, y1 - 5),
97                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 1)
98
99     # Display the image
100    plt.figure(figsize=(6, 6))
101    plt.imshow(img)
102    plt.axis("off")
103    plt.show()
104
105    # Call function to visualize training images
106    visualize_dataset("Train", num_images=5)
107    import os
108    import matplotlib.pyplot as plt
109    from collections import Counter
110
111    # Define dataset path
112    dataset_path = "/content/YOLO_dataset/Train"
113
114    # Get all class folders
115    class_counts = Counter()
116
117    for class_name in os.listdir(dataset_path):
118        class_path = os.path.join(dataset_path, class_name)
119
120        # Ensure it's a directory
121        if os.path.isdir(class_path):
122            images_count = len([f for f in os.listdir(class_path) if f.endswith((' .jpg', '.png', '.jpeg'
123                                         ))])
124            class_counts[class_name] = images_count
125
126    # Plot histogram
127    plt.figure(figsize=(12, 6))
128    plt.bar(class_counts.keys(), class_counts.values(), color="skyblue")
129    plt.xlabel("Class Name")
130    plt.ylabel("Number of Images")
131    plt.title("Class Distribution in YOLO Train Dataset")
132    plt.xticks(rotation=45, ha="right") # Rotate labels for better visibility
133    plt.grid(axis="y", linestyle="--", alpha=0.7)
134    plt.show()
135    %cd {HOME}
136
137    !python train.py \
138    --batch 16 --epochs 25 --img 640 --device 0 --min-items 0 --close-mosaic 15 \

```

```

138 --data /content/Formatted_dataset/dataset.yaml \
139 --weights weights/gelan-c.pt \
140 --cfg models/detect/gelan-c.yaml \
141 --hyp data/hyps/hyp.optimized.yaml
142 from IPython.display import Image
143
144 Image(filename=f'{HOME}/runs/train/exp2/results.png', width=1000)
145 %cd {HOME}
146
147 !python val.py \
148 --img 640 --batch 32 --conf 0.001 --iou 0.7 --device 0 \
149 --data /content/Formatted_dataset/dataset.yaml \
150 --weights {HOME}/runs/train/exp2/weights/best.pt
151 !python detect.py \
152 --img 1280 --conf 0.1 --device 0 \
153 --weights {HOME}/runs/train/exp2/weights/best.pt \
154 --source /content/Formatted_dataset/Test/images
155 import glob
156
157 from IPython.display import Image, display
158
159 for image_path in glob.glob(f'{HOME}/runs/detect/exp/*.jpg'):
160     display(Image(filename=image_path, width=600))

```

References

- [1] J. Chappidi; D. M. Sundaram(2024). Novel Animal Detection System: Cascaded YOLOv8 With Adaptive Preprocessing and Feature Extraction, IEEE Access, vol. 12, pp. 110575-110587.
- [2] K. M, B. V ; A. K(2024). Animal Intrusion Detection Using Yolo V8, 2024 10th International Conference on Advanced Computing and Communication Systems (ICACCS), pp.206-211.
- [3] Y. Zhang; Y. Lu; W. Zhang; X. Liu(20204). The efficient detection and supervision for safety risk operation of power substation based on the improvement of YOLOV5, 6th International Conference on Energy Systems and Electrical Power (ICESEP), pp.599-603.
- [4] A. P E; A. C K; M. Shibili; R. C K; N. K(2023). Development and Implementation of an Animal Intrusion Detection System Using Image and Audio Processing, 14th International Conference on Computing Communication and Networking Technologies (ICCCNT), pp.1-7.
- [5] R. Kothai; J. S. M; S. S; A. M; K. J(2023). Animal Intrusion Detection and Crop Protection Using Yolo and Sound Devices, Intelligent Computing and Control for Engineering and Business Systems (ICCEBS), pp.1-10.
- [6] M. V. K. Rao; A. Challawar; B. Adithi; B. Adithi; P. Bussa(2023). Enhancing Farm Security: Animal Intrusion Detection using YOLO, 2nd International Conference on Automation, Computing and Renewable Systems (ICACRS), pp.1476-1481.

- [7] J. J. Daniel Raj; C. N. Sangeetha; S. Ghorai; S. Das, Manish; S. Ahmed(2023). Wild Animals Intrusion Detection for Safe Commuting in Forest Corridors using AI Techniques, 3rd International Conference on Innovative Practices in Technology and Management (ICIPTM), pp.1-4.
- [8] A. D. Shetty; S. Ashwath(2023). Animal Detection and Classification in Image Video Frames Using YOLOv5 and YOLOv8, 7th International Conference on Electronics, Communication and Aerospace Technology (ICECA), pp.677-683.
- [9] R. Sumathi; P. Raveena; P. Rakshana; P. Nigila; P. Mahalakshmi(2022), Real Time Protection of Farmlands from Animal Intrusion, IEEE World Conference on Applied Intelligence and Computing (AIC), pp.859-863.
- [10] P. Muragod; V. R. K(2022), Animal Intrusion Detection using various Deep Learning Models, IEEE North Karnataka Subsection Flagship International Conference (NKCon), pp.1-12.